

Section 3.3 — Video Processing & Media Delivery at Scale

Enhanced Study Notes (Architecture-Heavy Section)

Study Directive: Give extra weight to before/after architecture diagrams and production topology comparisons between named systems (e.g., YouTube vs Netflix transcoding pipelines, HLS vs DASH vs CMAF, throughput-based vs buffer-based ABR).

Executive Overview

Video processing combines compute-intensive operations with massive scale. Understanding codec internals, adaptive bitrate (ABR) algorithms, and CDN integration is essential for designing platforms like YouTube or Netflix. A single 2-hour 4K movie upload generates ~5,000 individual files after transcoding.

1. Video Transcoding Pipeline Architecture

1.1 The Problem at Scale

Before (Naive Single-Pass Transcoding):

```
Upload: 10GB raw video file
|
▼ [Single Worker: decode → encode] ← takes 4 hours
|
▼ Output: 1 file (1080p H.264)
```

Problems:

- 4-hour time-to-live is unusable for live or viral content
- 1 quality = no adaptive bitrate → bad on mobile
- Single worker = no parallelism
- Failure = redo everything

After (Parallel Segmented Pipeline):

Upload: 10GB raw video file

— [Split into 60-second chunks → 120 segments]

— [120 GPU workers encode in parallel]

Each segment → all quality levels simultaneously
1080p@5Mbps + 720p@3Mbps + 480p@1.5Mbps + 360p@750Kbps

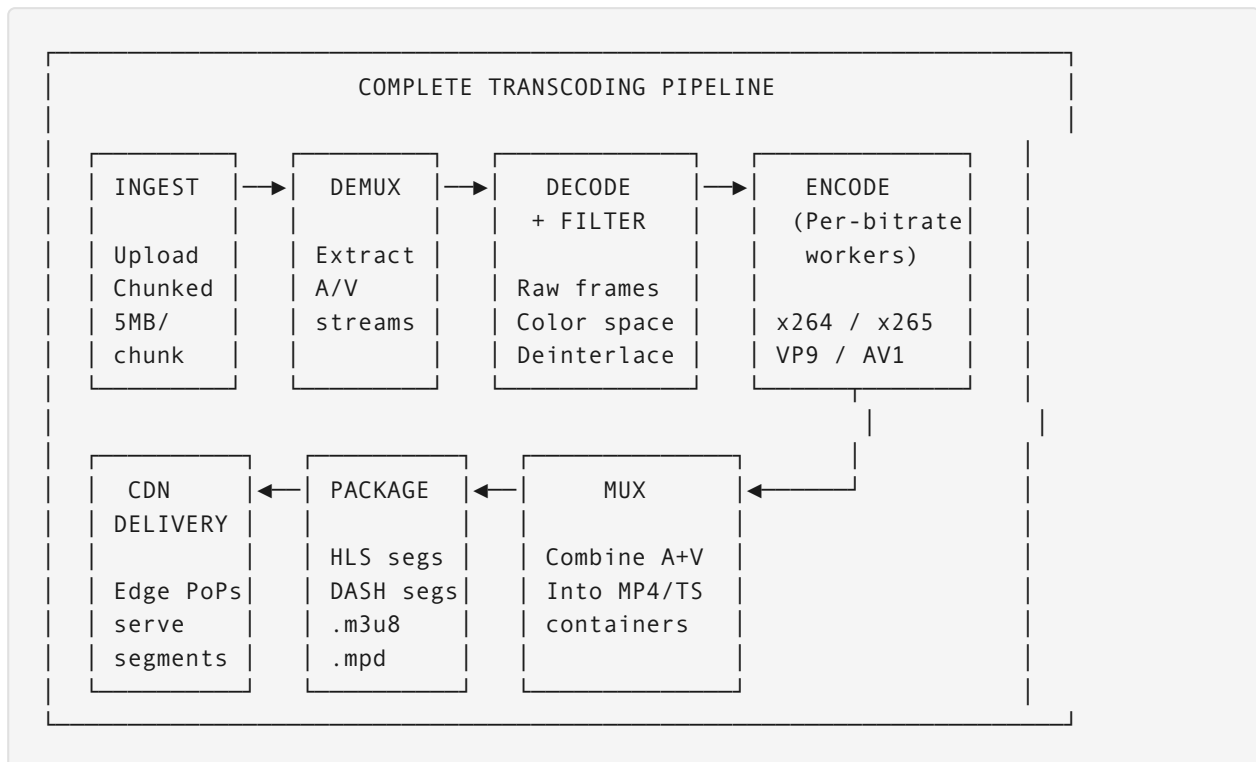
— [Stitch + package → HLS/DASH manifests]

Time-to-360p: 2 minutes (quick preview)

Time-to-1080p: 10 minutes (full ladder)

vs. 4 hours for single-pass

1.2 Complete Pipeline Stages



1.3 Decoding Internals

H.264 Frame Types Explained:

I-frame (Intra-coded):

Complete image, no reference to other frames.

Large size: ~100KB for 1080p frame

Used for: Seek points, scene changes, GOP boundaries

Decode time: Longest (full image)

P-frame (Predictive):

Encodes DELTA from previous frame.

Size: ~10KB (only what changed)

Example: Walking scene → background static → only person moves

Background: copied from previous frame (free!)

Moving parts: encoded as motion vectors + small deltas

B-frame (Bidirectional):

Encodes DELTA from previous AND next frame.

Size: ~5KB (most efficient)

Example: camera pan → can interpolate from both sides

Problem: Decoder needs future frame → buffering required

Trade-off: Higher compression, higher decode latency

GOP (Group of Pictures) structure:

I B B P B B P B B P B B I B B P ...

↑

Keyframe

(seek point)

↑

Next keyframe

GOP size trade-off:

Large GOP (10s): Better compression (fewer keyframes)

Slow seeking (must decode from last keyframe)

Small GOP (2s): Faster seeking (seek to nearest keyframe)

Larger file (more keyframes)

For ABR streaming: 2-4 second GOPs (segment alignment)

For low latency: 1-2 second GOPs

1.4 Bitrate Ladder Design

Adaptive Bitrate Ladder (Netflix Standard):

Resolution	Bitrate	Codec	Use Case
4K (2160p)	15-25Mbps	H.265	4K TV, fiber
1080p	4.3-8Mbps	H.264	Desktop, good WiFi
720p	2.5-4Mbps	H.264	Laptop, decent WiFi
480p	1.5Mbps	H.264	Mobile on WiFi
360p	750Kbps	H.264	Mobile data
240p	375Kbps	H.264	3G fallback
144p	150Kbps	H.264	Extreme fallback

Codec choice:

H.264 (AVC): Universal support, hardware decode everywhere
Royalty fees (MPEG-LA patent pool)

H.265 (HEVC): 50% better compression vs H.264
Same quality at half the bitrate!
Limited hardware support on older devices
Higher royalty costs

VP9 (Google): Royalty-free, competitive with H.265
Used by YouTube for web delivery

AV1 (AOM): Best compression (~30% better than H.265)
Royalty-free
Slow encode (10x slower than H.264)
Newer hardware support (2022+)
Used by Netflix for 4K+ streams

2. Adaptive Bitrate Streaming (ABR)

2.1 HLS vs DASH — Architecture Comparison

HLS Structure:

```
Master Playlist (video.m3u8):
#EXTM3U
#EXT-X-VERSION:6
#EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080,CODECS="avc1.640028"
1080p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1280x720,CODECS="avc1.4d001f"
720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=750000,RESOLUTION=640x360,CODECS="avc1.42001e"
360p/playlist.m3u8

Media Playlist (720p/playlist.m3u8):
#EXTM3U
#EXT-X-VERSION:6
#EXT-X-TARGETDURATION:4
#EXT-X-MAP:URI="init.mp4"           ← Initialization segment (codec info)
#EXTINF:4.000,
segment-0.ts                        ← MPEG-TS format, 4 seconds, ~1.5MB
#EXTINF:4.000,
segment-1.ts
#EXT-X-ENDLIST                     ← Absent for live streams (playlist updates)

HLS segments: .ts (MPEG-TS) or .m4s (fMP4 with CMAF)
```

DASH Structure:

```
Manifest (video.mpd):
<MPD type="static" mediaPresentationDuration="PT2H">
  <Period>
    <AdaptationSet mimeType="video/mp4" codecs="avc1.640028">
      <Representation id="1" bandwidth="5000000" width="1920" height="1080">
        <SegmentTemplate timescale="90000" media="1080p/segment-$Number$.m4s"
          initialization="1080p/init.mp4">
```

```

    <SegmentTimeline>
      <S t="0" d="360000" r="1799"/> ← 4s segments × 1800 = 2 hours
    </SegmentTimeline>
  </SegmentTemplate>
</Representation>
<Representation id="2" bandwidth="3000000" width="1280" height="720">
  ...
</Representation>
</AdaptationSet>
</Period>
</MPD>

```

HLS vs DASH Comparison:

Feature	HLS	DASH
Origin	Apple (2009)	MPEG (2012)
iOS/Safari support	✓ Native	✗ Needs Media Source Extensions
Android/Chrome	✓	✓ Native
Segment format	.ts or .m4s (fMP4)	.m4s (fMP4)
Live streaming	✓	✓
Low latency	LLHLS (LL-HLS)	CMAF + chunked transfer
DRM	FairPlay	Widevine / PlayReady
Typical latency	6–30s	6–30s (2–5s with low-latency)

CMAF — The Unifier:

Problem: HLS uses .ts segments, DASH uses .m4s segments.
 Two separate transcoding jobs to support both!
 Storage: 2x segments on CDN.

CMAF (Common Media Application Format):
 Uses fMP4 (.m4s) for BOTH HLS and DASH.
 Same segment files, different manifests.

CDN storage:
 Before CMAF: $2 \times (\text{segments} + \text{manifests}) = 2x \text{ cost}$
 After CMAF: $1 \times \text{segments} + 2 \times \text{manifests} \approx 1x \text{ cost}$

Production: Netflix, Disney+, Hulu all use CMAF.

2.2 ABR Algorithms — Detailed Comparison

Throughput-Based (BOLA predecessor):

Algorithm:

```
window = last 10 seconds of downloads  
throughput_bps = total_bytes_downloaded / window_time_seconds
```

Select highest quality where $\text{bitrate} < \text{throughput} \times \text{safety_factor}(0.8)$

Example:

```
Measured throughput: 5 Mbps  
Safety factor: 0.8  
Max usable: 4 Mbps  
Bitrate ladder: 5/3/1.5/0.75 Mbps  
Selected: 3 Mbps (720p) ← highest ≤ 4 Mbps
```

Problem — Network variability:

```
Bursty 4G: 2 Mbps for 2s, then 10 Mbps for 2s  
Average: 6 Mbps → selects 5 Mbps (1080p)  
During 2 Mbps burst: 5 Mbps segment takes 2.5 seconds to download  
Download time > segment duration → buffer empties → stall!
```

User sees: quality spike, then buffering = terrible experience

Buffer-Based (BOLA):

Algorithm:

Buffer monitor:

```
High buffer (> 30s): Increase quality (aggressive)  
Medium buffer (10-30s): Maintain quality  
Low buffer (5-10s): Decrease quality  
Critical buffer (< 5s): Emergency drop to lowest quality
```

Quality decision:

```
target_quality = f(buffer_level)
```

Example:

```
Buffer: 45 seconds → select 1080p (maximum)  
Buffer: 8 seconds → drop to 360p (emergency)  
Buffer: 20 seconds → maintain current quality
```

Problem — Slow reaction to throughput drops:

```
User on train enters tunnel (throughput → 0)  
Buffer: 25s → BOLA thinks "maintain quality"  
Downloads 1080p segment → takes 60 seconds on slow link  
Buffer depletes while waiting → stall  
BOLA reacts 25 seconds too late
```

Hybrid (BOLA-E / Netflix Pensieve):

Combines both signals:

```
throughput_prediction = weighted_average(recent_segments)
buffer_level = current_buffer_seconds
```

```
quality_score(q) = throughput_utility(q) × buffer_weight(q)
```

Throughput component: "Can I download at this quality?"

Buffer component: "Should I be conservative or aggressive?"

Hysteresis: Don't switch quality unless >20% difference
Prevents rapid oscillation (annoying to watch)

Pensieve (Netflix ML-based):

Trained on millions of viewing sessions

Input features: [past_throughputs, buffer_level, past_qualities,
chunk_size_predictions, time_in_video]

Output: Quality level for next segment

Result: 10-20% fewer stalls than BOLA

2.3 ABR Trade-Off Matrix

Algorithm	Stall Rate	Quality Stability	Bandwidth Efficiency	Complexity
Throughput-Based	Medium	Poor	Good	Low
Buffer-Based (BOLA)	Low	Good	Medium	Low
Hybrid	Low	Good	Good	High
ML-Based (Pensieve)	Lowest	Best	Best	Very High

3. Media CDN Integration

3.1 Upload Pipeline Architecture

Chunked Upload for Reliability:

```
Client (browser/app):
  file = select_file() # 10GB raw video

  # Split into chunks
```

```

chunks = split_into_5MB_chunks(file) # → 2048 chunks

# Compute checksums for integrity
for chunk in chunks:
    chunk.hash = sha256(chunk.data)

# Upload in parallel (configurable concurrency)
await asyncio.gather(*[
    upload_chunk(upload_id, i, chunk)
    for i, chunk in enumerate(chunks)
    if not already_uploaded(upload_id, i) # Resume support!
], concurrency=10) # 10 parallel uploads

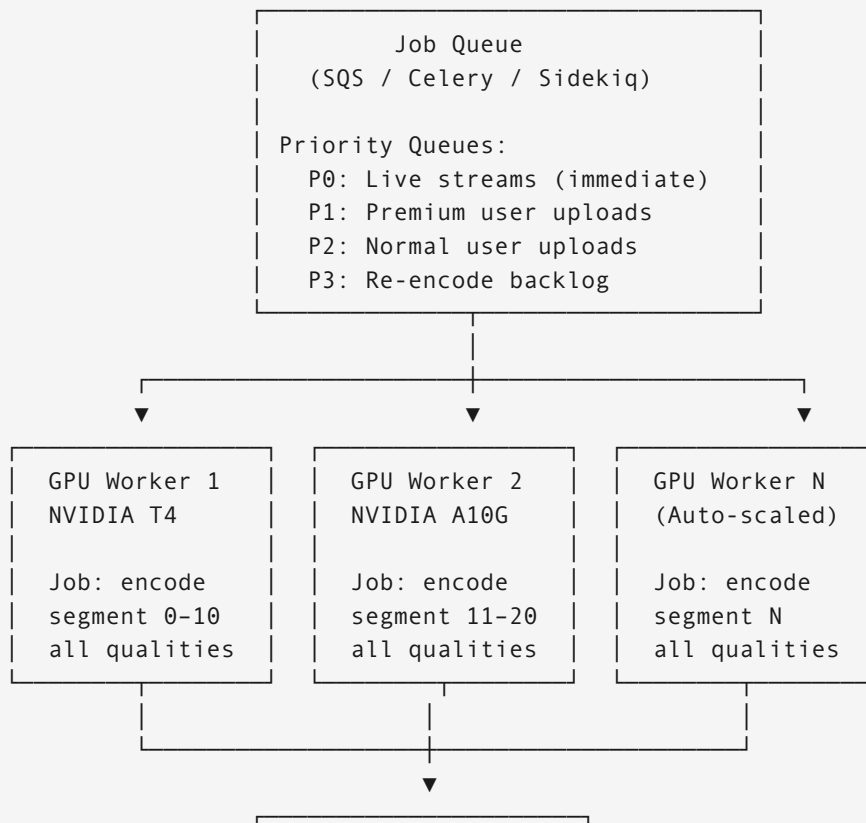
# Signal completion
await finalize_upload(upload_id)

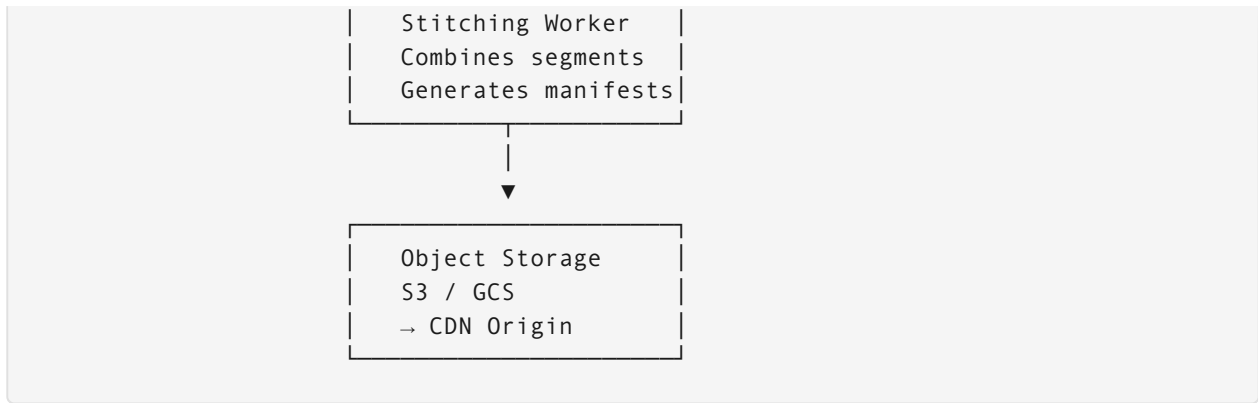
Server:
# Verify all chunks received with correct hashes
# Reassemble into single file (or keep as chunks for transcoding)
# Trigger encoding job queue

Resume support:
Client stores upload_id locally
On reconnect: GET /upload/{id}/status → [0,1,5,6 missing]
Re-upload only missing chunks

```

Transcoding Farm Orchestration:





3.2 Storage Architecture

Metadata vs Blob Separation:

WRONG (common mistake): Store everything in one database
videos table: id, title, raw_video_blob, segments_blob, ...

Problems:

- PostgreSQL row with 50GB video → absurd
- Can't replicate DB without replicating all video data
- No CDN-friendly access patterns

RIGHT: Metadata + Blob separation

Metadata Store (PostgreSQL / MySQL):

```
videos: {
  id: uuid,
  title: string,
  creator_id: uuid,
  duration_seconds: int,
  status: enum(processing|ready|failed),
  thumbnail_url: string, ← just URL, not blob
  created_at: timestamp,
  view_count: bigint
}
```

```
video_qualities: {
  video_id: uuid,
  quality: enum(1080p|720p|480p|360p),
  manifest_url: string,
  size_bytes: bigint,
  status: enum(processing|ready)
}
```

Metadata: ~1KB per video → millions of videos = GBs (manageable)

Blob Store (S3 / GCS):

Structure:

```
s3://media-bucket/
  videos/{video_id}/
```

```

raw/original.mp4          ← 10GB raw upload
hls/
  master.m3u8              ← 1KB manifest
  1080p/
    playlist.m3u8          ← 20KB (1 per segment listing)
    segment-0000.ts        ← 2.5MB × 1800 segments = 4.5GB
    segment-0001.ts
    ...
  720p/
    ...
  360p/
    ...
dash/
  video.mpd
  1080p/
    init.mp4
    segment-0000.m4s
    ...
thumbnails/
  thumb-{timestamp}.jpg    ← 10-100 thumbnails per video

```

Total per 2-hour video: ~20GB (raw: 10GB + all transcoded: ~10GB)

YouTube: 500 hours uploaded per minute × ~20GB ≈ 600TB/minute ingestion

3.3 Pre-Warming Strategy

ML-Based Popularity Prediction:

Features fed to model:

- creator.subscriber_count
- creator.avg_view_count_last_30d
- video.upload_time (peak hours?)
- video.title_keywords (trending?)
- social.shares_in_first_minute
- social.comment_velocity

Prediction output:

popularity_score: 0.0 - 1.0

Action matrix:

Score ≥ 0.9 (predicted viral):

- Push to ALL edge PoPs immediately
- Pre-allocate 4x bandwidth headroom
- Enable real-time monitoring for quality issues

Score 0.5-0.9 (predicted popular):

- Push to top 10 regional PoPs (where creator's audience is)
- Standard monitoring

Score < 0.5 (predicted normal):

- Pull CDN (lazy load on first request)
- No pre-warming

→ Standard TTL

Cost optimization:

Pre-warming 1 video to all edges: ~\$50 in bandwidth + storage

Pre-warming 1% of uploads: saves ~50% on origin bandwidth for top content

Net savings: positive when video gets >10K views in first hour

4. Interview Question — Extended Solution

Your video platform serves 10M concurrent streams. How do you design the transcoding pipeline to minimize time-to-live for new uploads?

Solution Framework

Multi-Stage Progressive Pipeline:

Stage 1 — Preview (target: 90 seconds from upload):

Goal: Get SOMETHING playable ASAP

Quality: 360p only

Process:

- Transcode first 60 seconds at 1x realtime
- Publish partial manifest (first 15 segments)
- User can start watching while rest encodes

t=0s:	Upload starts	
t=30s:	First chunk available	
t=60s:	360p first 60s ready	
t=90s:	360p first 2min ready	← PUBLISH: user can watch now

Stage 2 — Full Ladder (target: 10 minutes):

Goal: Full quality for all users

Process:

- Parallel workers: 1080p + 720p + 480p simultaneously
- Workers pick up where Stage 1 left off
- Progressive manifest update: add qualities as available

t=2min:	360p full video ready	
t=4min:	480p full ready	← Manifest updated: 2 qualities
t=7min:	720p full ready	← Manifest updated: 3 qualities
t=10min:	1080p full ready	← Manifest updated: all qualities

Auto-Scaling Worker Pool:

```
# AWS MediaConvert / Custom GPU cluster
class TranscodeOrchestrator:
    def submit_job(self, video_id: str, priority: int):
        # Estimate queue depth
        queue_depth = self.queue.size()
        current_workers = self.worker_pool.count()

        # Scale out if needed
        estimated_wait = queue_depth / current_workers # seconds
        if estimated_wait > 60: # > 1 minute wait
            self.worker_pool.scale_out(
                count=min(queue_depth // 10, 100), # Add workers, cap at 100
                instance_type="g4dn.xlarge" # NVIDIA T4 GPU
            )

        # Priority routing
        if priority == 0: # Viral / promoted
            return self.gpu_cluster.submit_immediate(video_id)
        else:
            return self.queue.enqueue(video_id, priority=priority)

    def handle_failure(self, video_id: str, error: str, attempt: int):
        if attempt < 3:
            # Retry with exponential backoff + alternative codec
            delay = 2 ** attempt * 30 # 30s, 60s, 120s
            self.queue.enqueue_delayed(video_id, delay_seconds=delay)
        else:
            # Fall back: encode single quality (360p only)
            self.encode_fallback(video_id, quality="360p")
            self.alert_team(video_id, error)
```

Fast-Track for High-Priority Creators:

Detection:

creator.tier == "partner" OR video.predicted_views > 1M

Action:

1. Allocate dedicated GPU instance (not shared pool)
2. Skip queue entirely → immediate processing
3. Multi-region parallel encoding (US + EU simultaneously)
4. Pre-warm CDN immediately on completion

Trade-off:

Dedicated GPU: \$2/hour × 10 min = \$0.33/video

vs standard queue: \$0.05/video

Acceptable for creators generating >\$100/video in revenue

Key Insight: Time-to-live optimization requires accepting and exposing intermediate states.

Progressive manifests that start with 360p and upgrade qualities transparently deliver far better user

experience than waiting 10 minutes for a “complete” encode. The user perceives the video as available immediately, even if not at full quality yet.

5. Additional Interview Questions

Q2: Netflix serves different video qualities per device. How does the player decide which quality to start at (avoiding the initial low-quality thumbnail effect)?

Problem:

Player starts at lowest quality (conservative)
User sees blurry 240p for first 3–5 seconds
Then jumps to 1080p
Looks unprofessional / jarring

Solutions:

1. Throughput pre-estimation:
Before requesting first segment, run lightweight probe:
 - HTTP HEAD request to CDN (measure response time)
 - Use TCP connection establishment time to estimate bandwidth
 - Start at quality = $\text{estimated_bandwidth} \times 0.7$ (safety margin)
2. Stored throughput history:
Client SDK stores previous session's throughput for this network/ISP
"Last time on WiFi-SSID-X, average throughput was 15 Mbps"
Start at 1080p immediately
3. Server-Side Start Quality Hint:
CDN edge returns header: X-Recommended-Initial-Quality: 720p
Based on: client IP → ISP → average CDN throughput for that ISP
4. Client preloading:
Before user clicks play, prefetch first 2 segments at 1080p
(video thumbnail hover → prefetch starts)
When user clicks: 1080p buffer already full → instant HD playback

Used by: YouTube (hover-prefetch), Netflix (title card hover)

Q3: How would you handle a live stream that suddenly gets 10x more viewers than expected?

Normal state: 1,000 concurrent viewers → 1 Gbps CDN egress
Viral moment: 10,000 viewers in 60 seconds → 10 Gbps needed

CDN layer (automatic):

CDN is designed for horizontal scale – 10x barely noticeable
Anycast BGP routes new viewers to nearest healthy PoP

If one PoP saturates, BGP withdraws route → neighbors take over
CDN: handles automatically without intervention

Origin / ingest layer (potential bottleneck):

Problem: Origin still needs to serve CDN edge misses

Solution: Origin Shield collapses all misses

But: Live stream = NEW segments every 2-4 seconds

All edges need each new segment → fan-out from origin shield

Protocol: PUSH from origin to shield, PULL for edges

Origin pushes segment → Origin Shield

Origin Shield replicates to regional PoPs (proactively)

Edge nodes pull from regional (always warm)

Origin → Origin Shield: 1 connection × 6 Mbps = 6 Mbps (manageable)

Auto-scaling transcode:

Live transcoding is stateful (can't parallelize like VOD)

Horizontal scale: multiple encoder instances for different quality levels

Encoder A: produces 1080p stream

Encoder B: produces 720p + 480p streams

Encoder C: produces 360p + 240p streams

All synchronized to same timecode

Origin Shield stitches manifests